

Relazione del progetto JUno

Centra Tommaso
Di Palma Giovanni

Metodologie di Programmazione C.d.L Informatica A.A. 2025/2026

15 settembre 2026

Indice

1	Introduzione	1
1.1	Cos'è JUno?	1
1.2	Requisiti di base	2
2	Architettura del Progetto: package e Classi	2
2.1	Classi principali	2
2.2	Strumenti di Java	3
3	Giocare a JUno	4
3.1	Avvio del programma ed Inizio della partita	4
3.2	Configurazione Partita	5
3.3	Creazione dei Giocatori	6
4	Giocare a JUno	7
4.1	Storico Mosse	8
4.2	Fine Partita	9
4.3	Impostazioni	10
4.4	Salvataggi	11
4.5	Architettura e Scelte Implementative	11
4.6	Descrizione dei Metodi	11
5	Regole del Gioco e Impostazione	12
5.1	GameController, Partita e View	12
5.2	Le carte da Gioco	13
5.3	Challenge	14
5.4	Dichiarazione JUno	15
6	Strategie Bot	15
6.1	Bot Aggressivo	15
6.2	Bot Casuale	15
6.3	Bot Difensivo	16
7	Conclusioni	16
7.1	Criticità del Progetto	16

1 Introduzione

1.1 Cos'è JUno?

JUno è un progetto che mira a sviluppare il gioco di carte UNO ma in Java (da qui JUno), realizzato per il corso di Metodologie di Programmazione. Tale progetto è stato sviluppato da un gruppo formato da due studenti del corso di laurea in Informatica, Centra Tommaso e Di Palma Giovanni, durante l'anno accademico 2025/2026. Il progetto ricrea fedelmente le regole del gioco originale, la composizione del mazzo di gioco composto da 108 carte in totale, delle quali:

- Carte numeriche: 76 carte numerate da 0 a 9, suddivise in quattro colori (rosso, verde, blu e giallo), con due copie per ogni numero e colore, ad eccezione dello 0 che ha una sola copia per colore;
- Carte speciali: 24 carte speciali, suddivise in quattro colori (rosso, verde, blu e giallo), con due copie per ogni tipo di carta speciale e colore. Le carte speciali includono: Skip, Reverse e Draw Two;
- Carte jolly: 8 carte jolly, suddivise in due tipi (Wild e Wild Draw Four), con quattro copie per ogni tipo di carta jolly.

Lo scopo del gioco è quello di essere il primo giocatore a rimanere senza carte in mano, seguendo le regole del gioco originale e utilizzando le carte speciali per ostacolare gli avversari. E' possibile avviare una partita in due modalità: **Classica** o **A Punti**:

- Classica: il gioco termina quando un giocatore rimane senza carte in mano.
- A Punti: suddiviso in più round e il round termina quando un giocatore rimane senza carte in mano e vengono calcolati i punti in base alle carte rimanenti in mano agli altri giocatori. Il gioco termina quando un giocatore raggiunge un punteggio prestabilito.

Il gioco è pensato per essere giocato da un minimo di 2 fino a 6 giocatori, con la possibilità di giocare contro avversari umani o contro l'intelligenza artificiale del computer (spiegati nel loro paragrafo). Inoltre è possibile assistere a una partita in corso come spettatore, senza poter interagire con il gioco, osservando le mosse che i bot andranno a fare, tramite l'opzione **Modalità Solo Bot**. JUno presenta anche due regole alternative opzionali, che possono essere attivate o disattivate a seconda delle preferenze dei giocatori:

- **Regola Stacking**: consente ai giocatori di rispondere a una carta Draw Two o Wild Draw Four con un'altra carta dello stesso tipo, accumulando così l'effetto della penalità per il giocatore successivo. Ad esempio, se un giocatore gioca una carta Draw Two, il giocatore successivo può rispondere con un'altra carta Draw Two, costringendo il giocatore successivo a pescare quattro carte invece di due.
- **Regola Number Rush**: consente ai giocatori di giocare più carte dello stesso numero in una sola volta, ad eccezione che la prima carta giocata sia una mossa valida. Ad esempio, se un giocatore ha due carte con il numero 5 di colori diversi, può giocarle entrambe in un solo turno, a condizione che la prima carta giocata sia valida rispetto alla carta in cima al mazzo degli scarti.

Una volta decisa la modalità di gioco, se attivare o meno le regole opzionali, e il numero di giocatori, si può iniziare a giocare. Il gioco proseguirà fino a quando un giocatore non rimarrà senza carte in mano, decretando così il vincitore della partita.

1.2 Requisiti di base

Per poter compilare ed eseguire il progetto JUno, non è necessario installare nessuna libreria esterna, in quanto il progetto è stato interamente sviluppato con le librerie standard di Java. L'interfaccia grafica è stata realizzata utilizzando la libreria Swing, che è inclusa nel JDK di Java.

Per lo sviluppo del progetto è stato utilizzato l'ambiente di sviluppo Eclipse e la versione con la quale è stata sviluppata e testata è Java 21, quindi è anche la consigliata per eseguire il progetto. Per eseguire il progetto è sufficiente l'utilizzo di Java 14. Una volta scaricato il progetto basta avviare l'esecuzione del programma tramite la classe principale **Main**, situata all'interno del package **main**.

2 Architettura del Progetto: package e Classi

Il progetto è stato organizzato seguendo la struttura MVC (Model-View-Controller), che separa la logica di business, l'interfaccia utente e il controllo del flusso dell'applicazione. La struttura dei package è la seguente:

- **model**: contiene le classi che rappresentano il modello del gioco, come le carte, i giocatori e le regole del gioco.
- **view**: contiene le classi che gestiscono l'interfaccia utente, come le finestre di gioco, i pannelli e i componenti grafici.
- **controller**: contiene un'unica classe che gestisce il flusso del gioco, coordinando le interazioni tra il Model e la View.

Oltre ai package MVC, il progetto include anche un package **main**, che contiene la classe principale **Main** per avviare l'applicazione, e un package **utility**, che contiene le classi per la gestione degli assets, come le immagini per l'interfaccia grafica, e la classe che gestisce il salvataggio e il caricamento delle partite.

Viene inoltre utilizzato l'**Observer**, un pattern di progettazione che consente di notificare automaticamente le modifiche dello stato del model alla view, garantendo così un aggiornamento coerente dell'interfaccia utente, lo **Strategy Pattern** per la gestione delle strategie di gioco dei bot.

Per il salvataggio sono state utilizzate le librerie moderne NIO.2 di Java, che consentono di gestire in modo efficiente i file e le directory, e la serializzazione degli oggetti per salvare lo stato del gioco in un file binario.

2.1 Classi principali

Le classi principali del progetto sono le seguenti:

- **Partita**: la classe cardine del package model, che rappresenta una partita di JUno. Gestisce lo stato del gioco, i giocatori, le carte e le regole.
- **Giocatore**: una classe astratta che rappresenta un giocatore, sia umano che bot.

- **Carta:** rappresenta una carta del mazzo di JUno, ogni carta è immutabile e riceve i propri valori da due enumerazioni: **Colori** e **Valori**, che definiscono rispettivamente il colore e il valore della carta.
- **StrategiaBot:** un'interfaccia che definisce i metodi per implementare diverse strategie di gioco per i bot. Le classi che implementano questa interfaccia definiscono il comportamento dei bot durante la partita.
- **GameController:** la classe principale del package controller, che gestisce il flusso del gioco, coordinando le interazioni tra il Model e la View. Riceve gli input dell'utente dalla vista e aggiorna il modello di conseguenza, notificando la vista per aggiornare l'interfaccia utente.
- **FinestraPrincipale:** la classe principale del package view, contiene tutte le varie schermate del gioco.
- **PannelloTavolo:** la classe che rappresenta il tavolo di gioco, visualizzando le carte dei giocatori, il mazzo degli scarti e le informazioni sullo stato della partita.

2.2 Strumenti di Java

Per lo sviluppo del progetto sono stati utilizzati diversi strumenti e librerie che Java mette a disposizione.

La classe astratta **Giocatore** è stata implementata per rappresentare sia i giocatori umani che i bot, consentendo di gestire le interazioni con il gioco in modo uniforme. In questo modo si è fatto un utilizzo del Factory Pattern, che consente di creare oggetti senza specificare la classe concreta dell'oggetto da creare.

Le interfacce utilizzate all'interno del progetto sono molteplici. Quelle implementate da noi sono principalmente **StrategiaBot** e **ModalitàGioco** per la gestione delle strategie dei bot e delle modalità di gioco:

- **StrategiaBot:** definisce i metodi per implementare diverse strategie di gioco per i bot. Le classi che implementano questa interfaccia definiscono il comportamento dei bot durante la partita in base alla strategia scelta.
- **ModalitàGioco:** definisce la modalità di gioco della partita, consente di utilizzare il pattern Strategy per implementare diverse modalità di gioco, come la modalità classica e la modalità a punti, definendo le condizioni di vittoria in base alla modalità scelta.

Per quanto riguarda le interfacce base di Java, oltre ad Observer (implementata dal GameController) ed Observable (implementata dalla classe Partita), sono state utilizzate le interfacce **Serializable** per la serializzazione degli oggetti.

I generics sono stati utilizzati per la gestione delle collezioni di oggetti, come le liste dei giocatori e delle carte, consentendo di creare strutture dati flessibili e riutilizzabili, come `ArrayList<Giocatore>` e `ArrayList<Carta>` per la gestione dei giocatori e delle carte. Inoltre, per scorrere in modo efficiente e sicuro queste collezioni, si è fatto largo uso del costrutto **for-each**. Sfruttando internamente gli iteratori di Java, questo approccio

ha permesso di semplificare la sintassi e prevenire errori legati agli indici (*IndexOutOfBoundsException*).

Infine, una particolare attenzione è stata dedicata alla **gestione delle eccezioni** (tramite blocchi `try-catch`). In particolare, sono stati gestiti gli errori relativi al caricamento delle risorse grafiche esterne (assets dell'interfaccia). Nel caso in cui un file immagine non venga trovato nel percorso specificato, l'eccezione viene catturata in modo da non causare il crash dell'applicazione, garantendo che il programma possa comunque avviarsi e che la partita rimanga fruibile.

3 Giocare a JUno

3.1 Avvio del programma ed Inizio della partita

Per avviare il gioco, come già detto, è sufficiente eseguire la classe **Main** all'interno del package **main**, la quale utilizza **SwingUtilities.invokeLater()** per avviare l'interfaccia grafica in un thread separato, garantendo così la reattività dell'applicazione e prevenendo lag visivi ed eventuali crash. Una volta avviata l'interfaccia grafica, ci troveremo davanti alla seguente schermata:



Figura 1: Schermata iniziale del gioco JUno.

Tale schermata viene gestita dalla classe **PannelloMenu**, che estende la classe **JPanel**, la quale contiene il logo, i pulsanti per avviare una nuova partita, accedere alle impostazioni e uscire dal gioco.

La schermata presenta:

- Il **Logo** del gioco, che rappresenta il marchio JUno e viene visualizzato nella parte superiore della schermata.
- Un pulsante **Gioca** per avviare una nuova partita, che porta alla schermata di selezione delle varie opzioni per la creazione della partita.

- Un pulsante **Impostazioni** per accedere alle impostazioni, dove sarà possibile caricare una partita salvata.
- Un pulsante **Esci** per uscire dal gioco, che chiude l'applicazione.

3.2 Configurazione Partita

Cliccando su **Gioca** si accede alla schermata di selezione delle opzioni per la creazione della partita, gestita dalla classe **PannelloConfigurazione**, che estende la classe **JPanel**. La schermata si presenta con le seguenti opzioni:

The screenshot shows a configuration window with a light gray background. It is divided into three main sections:

- MODALITÀ VITTORIA**: Contains two radio buttons, "Classica" (selected) and "A Punti". To the right of "A Punti" is a label "Punti Vittoria:" followed by a numeric spinner set to "500".
- REGOLE ALTERNATIVE**: Contains three checkboxes, all of which are unchecked: "Stacking Carte", "Number Rush", and "Modalità Solo Bot".
- IMPOSTAZIONI GIOCATORI**: Contains the text "Massimo 6 Player" and two numeric spinners labeled "Umani :" and "Bot :", both set to "1".

At the bottom of the window are two buttons: "INDIETRO" and "AVANTI".

Figura 2: Schermata di configurazione della partita.

Vediamo nel dettaglio le opzioni disponibili per la creazione della partita:

- **Modalità Vittoria:** Consente di scegliere tra la modalità classica e la modalità a punti, che determina le condizioni di vittoria della partita, inoltre spuntando la casella **A Punti** si potrà scegliere il numero di punti necessari per vincere dal menù a tendina. il valore base è 500, ma è possibile scegliere fra 250, 500 (default), 750 e 1000 punti.
- **Regole Alternative:** Consente di attivare le regole opzionali Stacking e Number Rush, che modificano le dinamiche di gioco. Le regole sono disattivate di default. Inoltre è possibile decidere se avviare il gioco in **Modalità Solo Bot** assistendo a una partita fra bot senza poter interagire con il gioco.
- **Impostazioni Giocatori:** Consente di scegliere il numero di giocatori umani e bot, con un minimo di 2 e un massimo di 6 giocatori totali dai loro menù a tendina. Inoltre si può notare che portando il valore dei giocatori umani a 0, verrà automaticamente spuntata la casella **Modalità Solo Bot**. Se verrà inserito un numero di giocatori non valido, verrà notificato il seguente errore:

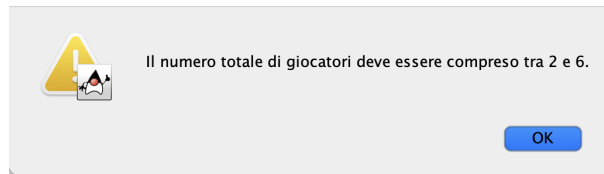


Figura 3: Errore per numero di giocatori maggiore di 6.

- **Indietro:** Consente di tornare alla schermata principale del gioco, senza salvare le impostazioni selezionate.
- **Avanti:** Consente di passare alla prossima schermata dove sarà possibile definire meglio le specifiche dei giocatori.

3.3 Creazione dei Giocatori

Dopo aver selezionato le varie modalità, regole e il numero di giocatori, cliccando su **Avanti** si accede alla schermata di creazione dei giocatori, gestita dalla classe **PannelloNomi**, che estende la classe **JPanel**.

La schermata si presenta così: È possibile notare che i campi per l’inserimento dei nomi

Figura 4: Schermata di creazione dei giocatori.

dei giocatori presentano dei valori preimpostati, che possono essere modificati a piacere.

Inoltre di fianco ai campi di inserimenti dei nomi per i Bot è possibile selezionare la strategia di gioco che il bot andrà ad utilizzare durante la partita, scegliendo tra le seguenti opzioni dall’apposito menù a tendina:

- **Casuale**
- **Aggressivo**
- **Difensivo**

Il comportamento dei vari bot è spiegato nel dettaglio nel paragrafo **Strategie dei Bot**.

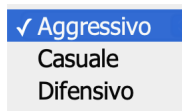


Figura 5: Menù a tendina per la selezione della strategia dei bot.

4 Giocare a JUno

Una volta inseriti i nomi dei giocatori e selezionate le strategie dei bot, cliccando su **Avvia Partita** si è pronti a giocare con le impostazioni selezionate.

Verrà mostrata la seguente schermata, gestita dalla classe **PannelloTavolo**, che estende la classe **JPanel**:



Figura 6: Schermata del tavolo di gioco.

Vediamo nel dettaglio i vari elementi presenti nella schermata del tavolo di gioco:

- **Tavolo di gioco:** contiene il mazzo di pesca (per pescare basta cliccare sul mazzo), la pila degli scarti, le impostazioni e il **Contesta JUno** (il funzionamento verrà spiegato nel paragrafo Regole del Gioco)



- **Turno Attuale:** la scritta mostra il nome del giocatore di turno

Turno di: Umano 1

- Turno Successivo: la scritta mostra il nome del prossimo giocatore di Turno

Prossimo Giocatore: Umano 2

- Numero Giocatori: viene mostrato il numero dei Giocatori

Giocatori: 6

- **Carte in Mano:** sono le carte che il giocatore ha in mano durante il suo turno. Per questioni di privacy e permettere un gioco equo su schermo condiviso, è stato inserito il tasto **Mostra Carte** (presente solo se ci sono più di due giocatori umani). Vicino alla disposizione delle carte in mano, troviamo il logo di JUno che una volta premuto dichiarerà lo stato di "JUNO" del giocatore (anche questa parte è dettagliata più avanti)

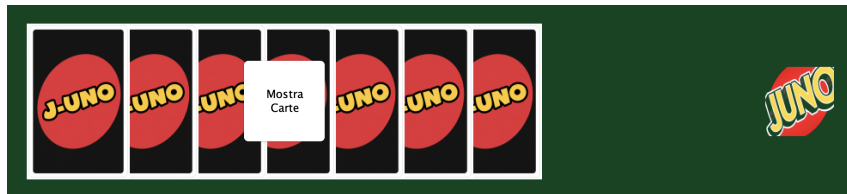


Figura 7: Visualizzazione carte senza Number Rush



Figura 8: Visualizzazione carte con Number Rush attivo

Si può notare un pulsante **Gioca** vicino al logo di JUno, servirà a giocare una o più carte in caso la regola Number Rush venga attivata. L'effetto di scelta delle carte è definito **hover dinamico** e richiama il gioco Balatro.

4.1 Storico Mosse

Quando si effettueranno delle mosse, è possibile notare delle scritte apparire vicino al tasto **Contesta JUno**, quelle scritte in realtà sono gestite da una classe definita come **PannelloStorico**, che estende la classe **JPanel**, è possibile consultarlo scrollandolo. Appare in questo modo: Lo storico contiene le seguenti informazioni:

- Ultima mossa eseguita da uno dei giocatori: Carta giocata o se il giocatore precedente ha pescato
- Le carte in mano all'ultimo giocatore

- Eventuali cambi colore



Inoltre a fine partita sarà possibile consultarlo per intero ma entreremo nel dettaglio al prossimo paragrafo.

4.2 Fine Partita

Una volta giocata la nostra partita saranno visualizzate due possibili schermate gestite dalla classe **PannelloClassifica**, che estende la classe **JPanel**, e appariranno così:

- **Vittoria Modalità Classica:** Al centro della schermata apparirà il nome del vincitore ed è possibile osservare la presenza anche dei seguenti pulsanti: **Menu** che permette di tornare al menù principale, **Nuova Partita** che inizierà una nuova partita con le stesse regole selezionate e **Storico** per consultare lo storico.



- **Vittoria Modalità A Punti:** Oltre ai tasti presenti è possibile notare la Classifica di fine Round e il tasto **Continua Partita**



Figura 9: Schermata di fine round



Figura 10: Schermata di fine partita a punti

Cliccando sull'apposito pulsante **Storico** presente in queste schermate, il giocatore ha la possibilità di ripercorrere l'intera partita appena conclusa. Verrà mostrata una schermata dedicata contenente la lista completa e sequenziale di tutte le mosse effettuate dai giocatori e dai bot, utile per analizzare l'andamento del gioco o verificare eventuali penalità e challenge.

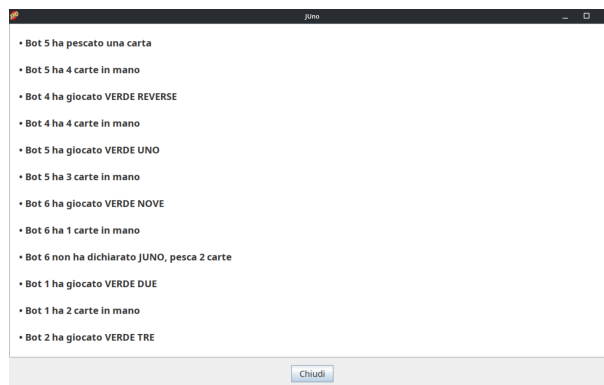


Figura 11: Schermata dello storico completo a fine partita

4.3 Impostazioni

Cliccando sull'ingranaggio si accede alle **Impostazioni**, gestite dalla classe **PannelloImpostazioni** che estende **JPanel**, vediamo come appare:



Figura 12: Impostazioni accessibili dal tavolo della partita



Figura 13: Impostazioni base accessibili dal menù base.

4.4 Salvataggi

La classe `GestoreSalvataggi`, situata nel package `utility`, è responsabile della gestione del salvataggio e del caricamento dello stato della sessione di gioco su supporto di memoria di massa. Progettata secondo il pattern *Utility Class*, presenta un costruttore privato e metodi statici per impedire l'istanziamento ed evitare modifiche allo stato interno.

4.5 Architettura e Scelte Implementative

L'architettura della classe si basa sul meccanismo di **serializzazione nativa di Java**, permettendo di convertire l'intero grafo di oggetti facenti capo alla classe `Partita` (incluso lo storico delle giocate) in una sequenza fluida di byte trasmissibile su disco.

Per la gestione dell'I/O sono state adottate le seguenti best-practice:

- **Java NIO.2** (`java.nio.file`): Utilizzo delle interfacce moderne `Path` e `Files` per un'interazione più efficiente e portabile con il file system sottostante rispetto alle API classiche.
- **Pattern Try-With-Resources**: Implementazione del meccanismo di gestione automatica delle risorse. Gli stream di input e output vengono istanziati nel blocco `try` per garantire la chiusura certa dei descrittori di file ed evitare *resource leak*, anche in caso di eccezioni a runtime.
- **Buffering delle Operazioni di I/O**: Inserimento di `BufferedOutputStream` e `BufferedInputStream` nella catena di decorazione degli stream. Ciò riduce drasticamente le operazioni di accesso fisico al disco (operazioni di I/O bloccanti), aggregando i dati in memoria temporanea prima della scrittura/lettura.

4.6 Descrizione dei Metodi

```
public static void Salva(Partita partita, String percorsoAssoluto)
```

Accetta in ingresso l'istanza corrente della `Partita` e la stringa rappresentante il percorso di destinazione. Converte il percorso in un oggetto `Path`, incapsula lo stream di output (`ObjectOutputStream`) su un buffer e serializza l'oggetto tramite il metodo `writeObject()`.

```
public static Partita Carica(String percorsoAssoluto)
```

Dato il percorso del file di salvataggio, apre un `ObjectInputStream` con supporto al buffering, deserializza il flusso di byte mediante `readObject()` ed esegue il casting esplicito restitutivo all'oggetto di tipo `Partita`.

5 Regole del Gioco e Impostazione

5.1 GameController, Partita e View

Come menzionato in precedenza la classe *Partita* è la base del model, ma vediamola nel dettaglio.

La classe contiene un campo istanziato dalla classe **Tavolo** la quale contiene:

- **Mazzo**: crea un nuovo mazzo composto dalle 108 carte di gioco per poi distribuirne 7 a testa.
- **Pila Scarti**: crea una Pila degli scarti, dove la prima carta serve per effettuare i vari controlli delle validità delle mosse.

Per istanziare la classe *Partita* si ha bisogno dei seguenti parametri:

- **Numero Giocatori e/o Bot**: separati in due parametri del costruttore di *Partita*, definiscono quanti giocatori ci sono in partita.
- **Regole**: un oggetto della classe **Regole** il quale contiene come campi d'istanza quali regole alternative sono state scelte.
- **Modalità**: la modalità di gioco decisa.
- **Nomi**: un array di stringhe contenente i vari nomi dei giocatori
- **Strategie**: un array di Strategie da assegnare ai vari bot.

Inoltre la classe *partita* ha due valori di default per stabilire il senso di turnazione dei giocatori.

- **sensoCorrente**: definisce se la rotazione nel turno secondo la formula presente nel metodo `passaTurno`. Parte da 1 (senso orario), quando viene utilizzata una carta Reverse diventerà -1 (senso antiorario)
- **turnoCorrente**: Parte da 0 e tramite la seguente formula:

$$(turnoCorrente + sensoCorrente + numGiocatori) \% numGiocatori$$

in questo modo nel momento in cui l'ultimo giocatore dell'array avrà eseguito il turno con il calcolo $\%numGiocatori$, tornerà a 0, ricominciando il giro da capo.

Il **GameController** rappresenta il cuore dell'architettura MVC del progetto, fungendo da ponte tra il Model e la View. Le sue responsabilità principali si articolano nei seguenti punti:

- **Gestione della Navigazione (View)**: All'avvio, il controller inizializza la **FinestraPrincipale** e richiama i metodi appositi per visualizzare il menù di base. Successivamente, si occupa di gestire tutte le transizioni tra i vari pannelli (Configurazione, Nomi e Tavolo di Gioco) in risposta alla navigazione dell'utente.
- **Inizializzazione del Model**: Agisce da raccoglitore per le impostazioni scelte nelle schermate di setup. Preleva le decisioni relative a regole alternative, modalità di vittoria, numero di giocatori, nomi e strategie dell'IA, passandole al costruttore della classe *Partita* per generare la sessione di gioco corretta.

- **Observer Pattern:** Il controller opera in ascolto attivo. Registrandosi come *Observer* della **Partita**, riceve notifiche immediate per ogni cambiamento di stato del modello (avanzamento del turno, cambio colore, applicazione di penalità) e comanda alla View gli aggiornamenti visivi necessari per riflettere la situazione aggiornata.
- **Gestione degli Eventi di Gioco:** Intercetta gli input dell'utente umano provenienti dal pannello di gioco (es. cliccare su una carta per giocarla, pescare dal mazzo, dichiarare JUno o lanciare una contestazione). Il controller traduce questi eventi grafici in chiamate ai metodi della classe **Partita**, garantendo che le regole vengano applicate e il flusso del gioco continui fluidamente.

La **View**, composta dalle varie classi estese da **JPanel** (come menzionato nei capitoli precedenti), svolge un ruolo puramente visivo e di interazione. Le sue caratteristiche principali nel pattern MVC sono:

- **Rappresentazione dello Stato:** Visualizza graficamente i dati provenienti dal Model in tempo reale (es. aggiornamento della mano, pila degli scarti, indicatore del turno corrente).
- **Cattura degli Input:** Registra le interazioni fisiche dell'utente (click sulle carte, pressione dei bottoni "JUno" o "Contesta") e le delega istantaneamente al **GameController** tramite appositi *event listener*.
- **Aggiornamento Reattivo:** Si limita a recepire i comandi di aggiornamento impartiti dal Controller (spesso scaturiti dalle notifiche dell'Observer), effettuando il *repaint* dei componenti grafici per garantire la sincronizzazione tra regole interne e schermo.

5.2 Le carte da Gioco

Come detto ad inizio relazione il mazzo è formato da 108 carte ma vediamo nel dettaglio gli effetti che andranno ad eseguire

- **Carte Numeriche:** Non hanno particolari effetti. Ogni carta assegna punti in base al valore numerico su di essa.
- **Skip:** Fa saltare il turno al giocatore successivo. Vale 20 punti.
- **Reverse:** Cambia il senso di turnazione dell'array. Vale 20 punti.
- **Draw Two:** Fa pescare due carte al prossimo giocatore e gli fa saltare il turno. In caso di Stacking, si accumulano le carte da pescare. Vale 20 punti.
- **Wild:** Fa selezionare un nuovo colore al giocatore che ha utilizzato la carta. Vale 50 punti.



Figura 14: Pop-up scelta nuovo colore

- **Wild Draw Four:** Fa pescare al giocatore 4 carte, gli fa saltare il turno e scegliere un nuovo colore al giocatore che ha utilizzato questa carta ma il giocatore che ha subito un Draw Four può chiedere un **Challenge**. Vale 50 punti.

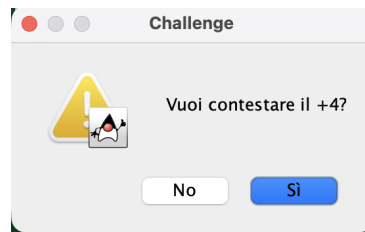


Figura 15: Pop-up per la scelta della contestazione

Le regole base di Uno dicono che una carta per essere valida come mossa, deve avere in comune o il colore o il valore numerico con la prima carta della Pila (ottenibile da Tavolo), per questi controlli ci viene in soccorso la classe Regole. All'interno della classe Regole è possibile trovare i seguenti metodi:

- **mossaValida:** richiede come argomento la carta giocata e la prima carta della pila degli scarti. Controlla se il colore o il valore combaciano tra la carta giocata e la carta della pila. In caso di Jolly verrà controllato solo il colore, impostato dal metodo di partita cambioColore.



Figura 16: Pop-up in caso di mossa non valida

- **applicaCarta:** una volta verificato che la carta utilizzata è valida vengono applicati gli effetti sopra citati.

5.3 Challenge

Come accennato nel paragrafo precedente il giocatore che ha subito un Draw Four può decidere di contestare il Draw Four.

Questo perché il Draw Four può essere utilizzato solo se non si hanno altre mosse in mano e in caso di utilizzo illegale è l'utilizzatore a subire la penitenza, in caso contrario invece se il Draw Four è stato utilizzato in modo corretto, il contestatore pescherà 6 invece di 4 carte.

Per permettere ciò abbiamo inserito all'interno della classe **Giocatore** un campo booleano chiamato **Challenge** che nel momento in cui il giocatore precedente ha utilizzato un Draw Four diventerà **true** facendo così scattare i vari controlli presenti in Partita.

5.4 Dichiarazione JUno

Analogamente al challenge, in **Giocatore** è presente un campo booleano chiamato **dichiarazioneJuno** che diventerà true nel momento in cui il giocatore premerà il logo di JUno nel Pannello Tavolo (figura 7).

Nel caso in cui un giocatore abbia esattamente una carta in mano potrà dichiarare "JUno" ma se non lo fa (volontariamente o meno), gli altri giocatori hanno la possibilità di contestarlo tramite il pulsante **Contesta JUno**, il quale controllerà se OGNI giocatore si è dimenticato di questa regola e in caso la risposta è positiva farà pescare 2 carte ai colpevoli.

6 Strategie Bot

Per la realizzazione del progetto è stato richiesto di creare il comportamento di 3 bot e i bot selezionati sono:

6.1 Bot Aggressivo

Questa strategia mira a ostacolare gli avversari il più velocemente possibile, privilegiando l'utilizzo immediato di carte ad effetto. Il suo comportamento si articola nei seguenti punti:

- **Scelta Carte:** Dà priorità assoluta alle carte speciali (Skip, Reverse, Draw Two, Wild). Solo in assenza di queste valuta la regola *Number Rush* o gioca la prima carta numerica valida trovata.
- **Contestazioni e JUno:** È un avversario spietato: se un giocatore dimentica di dichiarare JUno, il bot aggressivo lo contesterà *sempre* (100% di probabilità). Si ricorda di dichiarare JUno per sé stesso con un'affidabilità dell'85%.
- **Challenge (+4):** Rischia volentieri. Lancia una contestazione se ha la mano molto piena (≥ 7 carte), molto vuota (≤ 2 carte) o, in alternativa, affidandosi a un 25% di probabilità casuale.

6.2 Bot Casuale

L'intelligenza artificiale casuale non segue alcuna tattica specifica e si affida pesantemente alla generazione di numeri pseudo-casuali (**Random**). È imprevedibile per natura:

- **Scelta Carte:** Raccoglie tutte le mosse valide disponibili in quel turno e ne estrae una a caso. Presenta inoltre un 15% di probabilità di pescare volontariamente dal mazzo *anche se possiede carte giocabili*. Sfrutta il *Number Rush* solo nel 50% dei casi in cui è attivabile.
- **Colori e JUno:** Sceglie il nuovo colore dopo un Jolly in modo puramente casuale (25% per ogni colore). Dichiarare JUno nel 75% dei casi e contesta le mancanze avversarie nel 67% dei casi.
- **Challenge (+4):** La decisione di contestare o meno un Wild Draw Four è essenzialmente il lancio di una moneta (50% di probabilità).

6.3 Bot Difensivo

Progettato per minimizzare i danni e conservare le risorse per il *late game*, questo bot analizza attentamente il tavolo e calcola i rischi:

- **Scelta Carte:** Adotta una logica di conservazione. Se ha 2 o meno mosse valide ma ha almeno 3 carte in mano, preferisce *pescare* per accumulare opzioni difensive. Conserva il Wild Draw Four (+4) usandolo solo come estrema risorsa se non ha altre mosse. Quando gioca, analizza le mosse valide e predilige quelle del colore più frequente nella sua mano.
- **Cambio Colore:** A differenza degli altri bot, usa una `HashMap` per contare le occorrenze dei colori nella propria mano (esclusi i Jolly) e forza sempre il colore dominante.
- **Challenge (+4):** Valuta il peso della propria mano: se rischia un contraccolpo troppo oneroso (mano totale ≥ 60 punti), non contesta mai. Se invece il giocatore precedente gli lancia un +4 pur avendo ≥ 8 carte in mano, il bot assume che stia bluffando e lo contesta sistematicamente.
- **Attenzione al Gioco:** Mantiene un'alta concentrazione: dichiara JUno nel 95% dei casi e contesta attivamente le omissioni altrui (75% di probabilità).

7 Conclusioni

Il progetto è stato realizzato dai due studenti Centra Tommaso e Di Palma Giovanni, la divisione è stata equa ma ogni componente del gruppo ha partecipato allo sviluppo in modo attivo alla realizzazione di JUno.

Centra Tommaso si è occupato della gestione della View e del Controller, creazione del canale comunicativo (GitHub), stesura del UML e creazione dei vari aspetti grafici.

Di Palma Giovanni si è occupato della stesura della seguente relazione, la creazione del model e l'interpretazione delle varie AI.

7.1 Criticità del Progetto

A causa della rapida esecuzione concatenata dei metodi e della complessità dei thread in background, i log stampati nello storico in-game potrebbero talvolta risultare visivamente accavallati o non strettamente in ordine cronologico al millisecondo. Tutte le mosse vengono comunque registrate.